

W czasie przygotowania do kolokwium oraz podczas jego trwania wszystkie działania wykonywane na ekranie komputera są rejestrowane i mogą zostać odtworzone przez prowadzących. Po napisaniu kolokwium, katalog ze źródłami należy spakować do formatu zip lub tar.gz i wysłać go przy użyciu kampusu. Po wysłaniu plików proszę wyłączyć komputer.

Podczas kolokwium można korzystać materiałów:

1. dostępnych na kampusie w kursie programowania obiektowego,
2. zgromadzonych samodzielnie przed rozpoczęciem kolokwium.

Zabroniona jest wzajemna komunikacja oraz publikowanie i dostęp do kodu stworzonego na potrzeby kolokwium. Zabronione jest korzystanie z innych urządzeń niż udostępniony komputer.

Przedstawione poniżej kroki stanowią propozycję kolejności rozwiązywania zadania. Po przeczytaniu całości można zdecydować o rozwiązywaniu w innej kolejności. Widoczność metod należy określić samodzielnie zgodnie z zasadami enkapsulacji. Należy założyć, że dane wejściowe są poprawne.

Krok 1.

Utwórz rekord `Product` zawierający opis sprzedawanego produktu: `id` typu `int`, `name` typu `String`, `price` typu `int`.

Krok 2.

Zapoznaj się z dołączoną bazą SQLite zawierającą tabelę `product` — trzy pierwsze kolumny odpowiadają rekordowi `Product`, a czwarta kolumna `quantity` przechowuje ilość dostępnych sztuk w magazynie. Napisz klasę `DatabaseHandler`, która połączy się z tą bazą.

Krok 3.

W `DatabaseHandler` napisz trzy metody wykorzystujące zapytania SQL:

- `getProducts()` — zwraca listę wszystkich produktów z bazy:
`SELECT id, name, price FROM product.`
- `getQuantity(id)` — zwraca dostępną ilość dla podanego `id`:
`SELECT quantity FROM product WHERE id = ?.`
- `setQuantity(id, quantity)` — ustawia nową ilość dla podanego `id`:
`UPDATE product SET quantity = ? WHERE id = ?.`

Krok 4.

Stwórz klasę `Warehouse`, której konstruktor przyjmuje referencję do obiektu `DatabaseHandler`. Klasa powinna przechowywać informacje o dostępnych produktach i ich ilościach — pobierz je z bazy w konstruktorze. Dla uproszczenia, `Warehouse` nie przechowuje informacji o stanie konta.

W kolejnych krokach przekazuj referencję do obiektu `Warehouse` wszędzie tam, gdzie będzie to wymagane.

Krok 5.

Zapoznaj się z klasą `WebServer`. Zmodyfikuj jej metodę `buildHtmlPage()`, tak aby wyświetlała faktyczny stan magazynu — udostępnij potrzebne dane z obiektu `Warehouse`.

Krok 6.

W klasie `Warehouse` stwórz metodę `setDiscount(id, discount)`, która przyjmuje `id` produktu i procentową obniżkę (z obustronnie domkniętego zakresu od 0 do 100). Obniżona cena (zaokrąglona do liczby całkowitej) powinna być widoczna na stronie generowanej przez `WebServer`, ale nie powinna być zapisywana do bazy danych. Restart programu zeruje wszystkie obniżki.

Krok 7.

Zapoznaj się z klasą `DiscountWindow`. Zapełnij listę rozwijaną produktami z magazynu. Po naciśnięciu

przycisku wywołaj metodę `setDiscount()` z parametrami wprowadzonymi przez użytkownika za pomocą interfejsu graficznego.

Krok 8.

Stwórz klasę `Server` obsługującą równoległe połączenia TCP od wielu klientów. `Server` i `WebServer` powinny działać równocześnie na różnych portach.

Krok 9.

Stwórz klasę `Cart` reprezentującą koszyk zakupowy klienta. Koszyk — podobnie jak magazyn — może przechowywać produkty w różnych ilościach i cenach. `Cart` powinien dziedziczyć po klasie `ClientHandler`. Nadpisz jej metody abstrakcyjne (implementacja zostanie omówiona w kolejnych krokach) i wykorzystaj w serwerze do obsługi klientów.

Krok 10.

W klasie `Cart` zaimplementuj atomową metodę `handleAdd(id, quantity)` reprezentującą dodanie produktu do koszyka.

Jeśli magazyn dysponuje wystarczającą ilością, produkt powinien zostać zarezerwowany i dodany do koszyka — rezerwacja zmniejsza widoczną ilość na stronie `WebServer`, ale nie modyfikuje bazy danych. Serwer odsyła klientowi koszt ostatniej operacji: aktualną cenę zamówionego produktu pomnożoną przez `quantity`.

Jeśli produktów jest zbyt mało, klient otrzymuje stosowny komunikat, a koszyk pozostaje bez zmian.

Krok 11.

W klasie `Cart` zaimplementuj metodę `onLineReceived(line)`, tak aby interpretowała otrzymaną linię. Wysłanie przez klienta wiadomości `add <id> <quantity>` powinno spowodować wywołanie metody `handleAdd()`.

Krok 12.

W klasie `Cart` zaimplementuj metodę `handlePay()` reprezentującą zakup zawartości koszyka. Jeśli ceny zamówionych produktów nie zmieniły się od momentu ich dodania, serwer informuje klienta o kwocie do zapłaty, czyści koszyk i potwierdza transakcję. Zmiana stanu zapisywana jest w bazie danych. Metoda wywoływana jest w odpowiedzi na komendę `pay`.

Krok 13.

Jeśli między dodaniem produktów a wysłaniem komendy `pay` nastąpiła zmiana cen, klient powinien zostać o tym poinformowany i poproszony o potwierdzenie poprzez ponowne wysłanie komendy `pay`.

Krok 14.

Jeśli klient zerwie lub zakończy połączenie, wszystkie produkty z jego koszyka powinny wrócić do magazynu.